

CM05 - Libraries and Control Structures

I/O library functions, formats, conditions, switch, loops

Louis Ledoux

2026-07-05

Libraries and Basic I/O

Libraries and Basic I/O

Live pacing: about 8 min

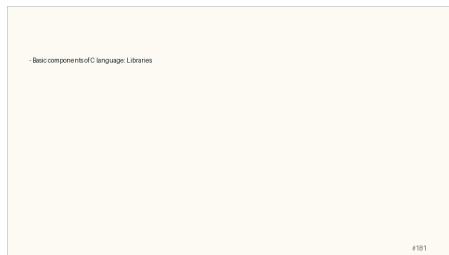
Question. How does a C program communicate and choose a path?

Core claim. Library calls and control structures turn values into observable behavior.

Target capability. Students can select the right format specifier, branch, and loop for a small task.

Main Path

- ▶ Start from the question: How does a C program communicate and choose a path?
- ▶ Build the model: Library calls and control structures turn values into observable behavior.
- ▶ End with a checkable action: Students can select the right format specifier, branch, and loop for a small task.
- ▶ Keep only this slide on the horizontal path during the live lecture.



Worked Example

- ▶ Use this as the live trace: Read one value, validate it, print a formatted result, then add a loop for repeated input.
- ▶ Representative source slide: 183
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.

```
Two basic I/O functions!

We have two main functions for read and write to the standard input and output
Write: int printf(format_t, a2, ..., an)
Read: int scanf(format_t, a2, ..., an)
To use them we need to add the directive
#include <stdio.h>
```

#183

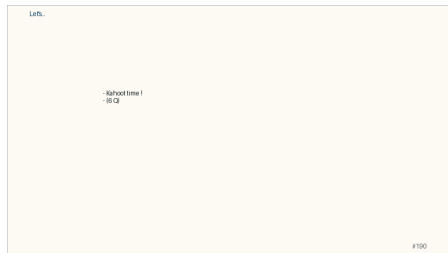
Anecdote Or Motivation

`printf` is a good warning: the compiler trusts the format string more than the value.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.



Backup Index

Original slides 181-190

- Basic components of C language: Libraries

Backup: Basic components of C language: Libraries

Original slide 181

- Basic components of C language: Libraries

Backup: Libraries

Original slide 182

In C, there are a lot of functions already implemented

Include system calls

Mathematical computations

...

The programmer can and should use them !

Add corresponding directive to the head of the compilation file

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <math.h>
```

```
#include <string.h>
```

...

Libraries

In C, there are a lot of functions already implemented into libraries

Include system calls

Mathematical computations

The programmer can and should use them !

Add corresponding directive to the head of the compilation file

#include <stdio.h>

#include <stdlib.h>

#include <math.h>

#include <string.h>

#182

Backup: Two basic I/O functions!

Original slide 183

```
We have two main functions for read and write to tl  
Write: int printf(format,e1,e2,...,en);  
Read: int scanf(format,a1,a2,...,an );  
To use them we need to add the directive  
#include <stdio.h>
```

Two basic I/O functions!

We have two main functions for read and write to the standard input and output
Write: `int printf(format,e1,e2,...,en);`
Read: `int scanf(format,a1,a2,...,an );`
To use them we need to add the directive
`#include <stdio.h>`

#183

Backup: Formated write: printf(...)

Original slide 184

Syntax

```
int printf(format,e1,e2,...,en);
```

Description

Writes (displays) the parameters $e_1, e_2, \dots, e_n$ to the standard output stream.

The format consists of I/O expressions and, potentially, escape sequences.

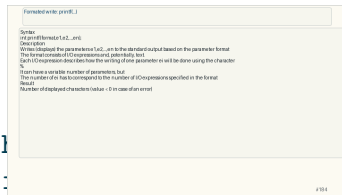
Each I/O expression describes how the writing of one parameter e_i will be done.

It can have a variable number of parameters, but

The number of e_i has to correspond to the number of I/O expressions specified in the format.

Result

Number of displayed characters (value < 0 in case of an error)



Backup: Formated read: scanf(...)

Original slide 185

Syntax

```
int scanf(format,a1,a2,...,an);
```

Description

Reads from the standard input (keyboard) the values

It assigns each read value to the variables, whose

Address of a variable is noted by the operand & &nom_variable

It can have a variable number of parameters, but

The number of ai has to correspond to the number of I/O expressions specifie

Result

Returns the number of elements correctly assigned



Backup: Format of I/O expressions

Original slide 186

%d: implies that the I/O is a **signed** integer (**int**)
%c : implies that the I/O is an ASCII character (**char**)
%f : implies that the I/O is a **float** (**float**).
%lf : implies that the I/O is a **double** (**double**).
%x: implies that the I/O is a **unsigned** integer to **hexadecimal**
%s : implies that the I/O is a string
....

More info: `man format` or google `printf`

Remarks

The format can also have normal text (whatever does not have the % character)



Backup: Example

Original slide 187

Example

```
int main()
{
    int i;
    float f;

    printf("Entrez un entier :");
    scanf("%d",&i);

    printf("Entrez un nombre réel :");
    scanf("%f",&f);

    printf("Vous avez saisi %d et %f \n", i, f);
}
```

Backup: What happens if we use a wrong format?

Original slide 188

- ▶ Usually we have only a compiler warning
- ▶ Question : What prints the following program?
- ▶ Answer: Segmentation fault (core dumped) or unexpected behaviour
- ▶ Tries to print a string of characters stored in the memory starting at the address 1654 and its end is indicated by the `\0` character.
- ▶ At the end, the program will try to access an illegal address
- ▶ Very common error!

What happens if we use a wrong format?

Usually we have only a compiler warning
Question: What prints the following program?
Answer: Segmentation fault (core dumped) or unexpected behaviour
Tries to print a string of characters stored in the memory starting at the address 1654 and its end is indicated by the `\0` character.
At the end, the program will try to access an illegal address
Very common error!

```
#include <stdio.h>
void main()
{
    int a = 1654;
    printf("%s", a);
}
```

#188

Backup: Type conversion: Inbuilt typecast functions

Original slide 189

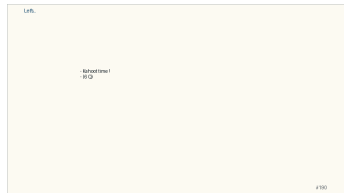
- ▶ `atof()` : convert string to float
- ▶ `atoi()` : convert string to integer
- ▶ `atol()` : convert string to long
- ▶ `itoa()` : convert integer to string
- ▶ `ltoa()` : convert long to string



Backup: Let's...

Original slide 190

- ▶ Kahoot time !
- ▶ (6 Q)



Control Structures

Control Structures

Live pacing: about 8 min

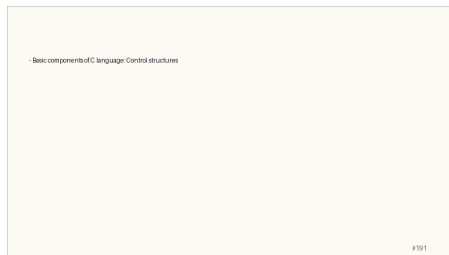
Question. How does a C program communicate and choose a path?

Core claim. Library calls and control structures turn values into observable behavior.

Target capability. Students can select the right format specifier, branch, and loop for a small task.

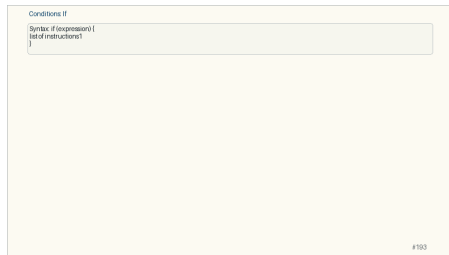
Main Path

- ▶ Start from the question: How does a C program communicate and choose a path?
- ▶ Build the model: Library calls and control structures turn values into observable behavior.
- ▶ End with a checkable action: Students can select the right format specifier, branch, and loop for a small task.
- ▶ Keep only this slide on the horizontal path during the live lecture.



Worked Example

- ▶ Use this as the live trace: Read one value, validate it, print a formatted result, then add a loop for repeated input.
- ▶ Representative source slide: 193
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.



Anecdote Or Motivation

`printf` is a good warning: the compiler trusts the format string more than the value.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.

Other types of objects

- Derived data types
- Pointer type
- Array type
- Structure type
- Union type
- Enumerated types
- Arithmetic types used to define variables
- Variables can only assign certain discrete integer values
- Function type
- Void type

#108

Backup Index

Original slides 191-198

- Basic components of C language: Control structures

Backup: Basic components of C language: Control structures

Original slide 191

- Basic components of C language: Control structures

Backup: Control structures

Original slide 192

- ▶ Similar to the ones in JAVA and Scripts
- ▶ NO BOOLEAN type!
- ▶ Example

```
int i=7;
while(i) {
  i--;
  ...
}

int i=7;
while(i!=0) {
  i--;
  ...
}
```

- ▶ Both ways are equivalent, BUT the second one is more readable!

Control structures

- Similar to the ones in JAVA and Scripts
- NO BOOLEAN type!
- Example

```
int i=7;
while(i) {
  ...
}

int i=7;
while(i!=0) {
  ...
}
```

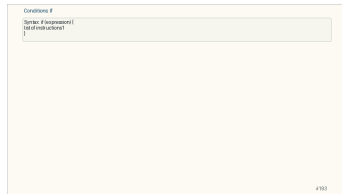
- Both ways are equivalent, BUT the second one is more readable!

#192

Backup: Conditions: If

Original slide 193

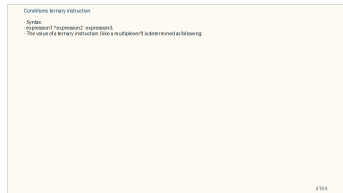
```
Syntax: if (expression) {  
list of instructions1  
}  
if (expression) {  
list of instructions1  
} else{  
list of instructions2  
}
```



Backup: Conditions: ternary instruction

Original slide 194

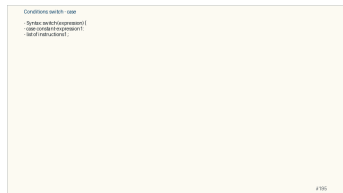
- ▶ Syntax:
expression1 ? expression2 : expression3;
- ▶ The value of a ternary instruction (like a multiplexer?) is determined as following:
- ▶ expression1 is evaluated.
- ▶ If it is true, then expression2 is evaluated and becomes the result of the entire expression.
- ▶ If it is false, then expression3 is evaluated and its value becomes the result of the expression.



Backup: Conditions: switch - case

Original slide 195

- ▶ Syntax: `switch(expression) {`
- ▶ `case constant-expression1:`
- ▶ `list of instructions1 ;`
- ▶ `break ; (optional)`
- ▶ `case constant-expression2: list of instruction2 ;`
- ▶ `break ;`
- ▶ `default: (optional)`
- ▶ `list of instructions3 ;`
- ▶ `}`



Backup: Loops

Original slide 196

```
Syntax while while (expression) {  
list of instructions ;  
}  
  
Syntax do-while do {  
list of instructions  
} while (expression) ;  
  
Syntax for for(expression1; expression2; expression3) {  
list of instructions ;  
}  
  
expression1: initialization  
expression2: condition of termination  
expression3: step
```



Backup: Loops: Control statements

Original slide 197

- ▶ Change execution from its normal sequence
- ▶ Instruction break;
- ▶ Terminates the loop or switch statement
- ▶ Transfers execution to the statement immediately following the END of the loop or switch.
- ▶ Instruction continue;
- ▶ Causes the loop to skip the remaining of its BODY
- ▶ Immediately retests the condition expression before reiterating.
- ▶ Instruction goto label;
- ▶ Recommendation: NEVER USE goto ! (except special cases)

Loop Control statements

- Change execution from its normal sequence
- Instruction break
- Terminates the loop or switch statement
- Transfers execution to the statement immediately following the END of the loop or switch.
- Instruction continue
- Causes the loop to skip the remaining of its BODY
- Immediately retests the condition expression before reiterating
- Instruction goto label
- Recommendation: NEVER USE goto ! (except special cases)

#197

Backup: Other types of objects

Original slide 198

- ▶ Derived data types:
- ▶ Pointer type
- ▶ Array type
- ▶ Structure type
- ▶ Union type
- ▶ Enumerated types:
- ▶ Arithmetic types used to define variables
- ▶ Variables can only assign certain discrete integer values.
- ▶ Function type
- ▶ Void type

